WigglyPaint by Umm..

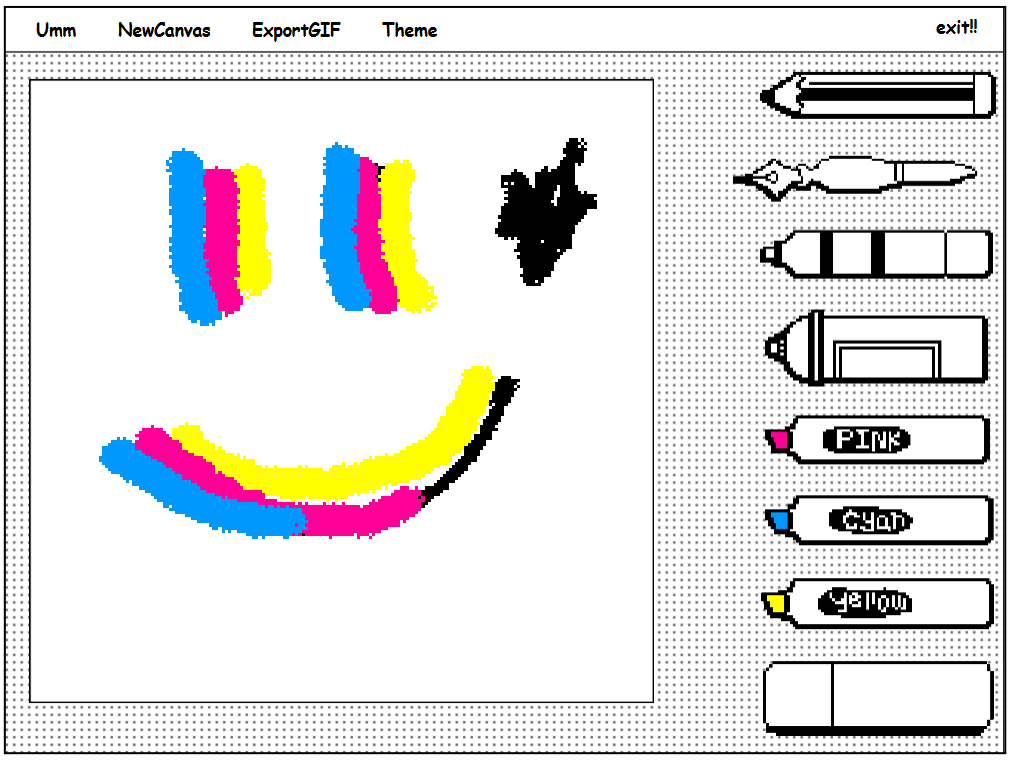
Karitt Thanawan 6730057621

Kaokanya Kokirdpanich 6730051821

1. Overview

WigglePaint is a JavaFX-based pixel-art drawing application designed around two main ideas: Ultra-simple retro painting workflow using pixel brushes, marker pens, highlighters, spray tools, and erasers. A unique “wiggle effect” rendering system that animates drawings by alternating between two visual states, giving the entire canvas a hand-drawn, shaky aesthetic (similar to old cartoons or wiggly-line animations).

The app uses a layered canvas architecture, custom brushes built on a Paintable interface, a CanvasData pixel array for crisp pixel-level drawing, and a dynamic theme system that recolors brush icons and UI elements.



2. Features

2.1 Drawing Tools

- Pencil Brush — Hard pixel brush with 1px size.
- Marker Pen — Adds jitter and soft edges to emulate a marker stroke.
- Fountain Brush — Pressure-like effect using speed scaling.
- Highlighters — Colorful strokes in 3 main colors; pink, cyan, and yellow.

- Spray Brush — Particle-style dot spraying within a radius.
- Eraser — Removes pixels from any layer.

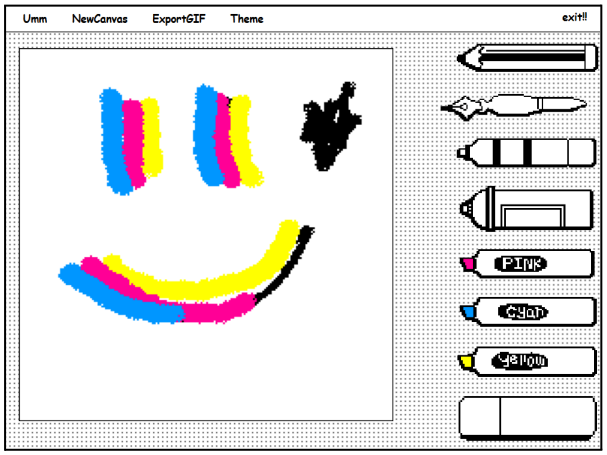
2.2 GIF Exporting

WigglePaint includes a built-in **Export to GIF** feature that transforms the app's signature **wiggle animation** into a lightweight animated GIF. The app internally maintains **two drawing frames** (Frame A and Frame B) that alternate to create the hand-drawn wiggly effect.

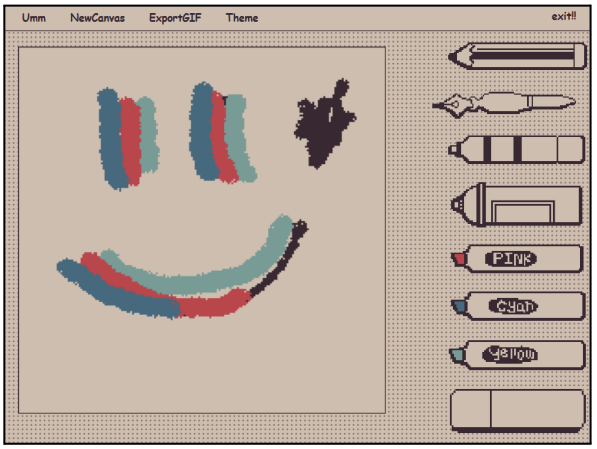
smile.gif

2.3 Theme System

WigglePaint allows the entire application—UI, background, and brush icons—to adapt instantly to different visual styles using a unified color palette. Each theme defines five colors: **Foreground**, **Background**, **Primary**, **Secondary**, and **Accent**. Instead of storing multiple icon sets, WigglePaint uses a smart recoloring engine (ImageThemeRender) that maps specific source colors in each PNG icon to the active theme colors, ensuring all tools match the selected palette while preserving clean pixel edges. When a theme is changed, the UI refreshes immediately: the menu bar, dot-pattern background, canvas grid, and icons all update together to create a coherent, consistent aesthetic. This system makes it easy to support classic, paste, etc.



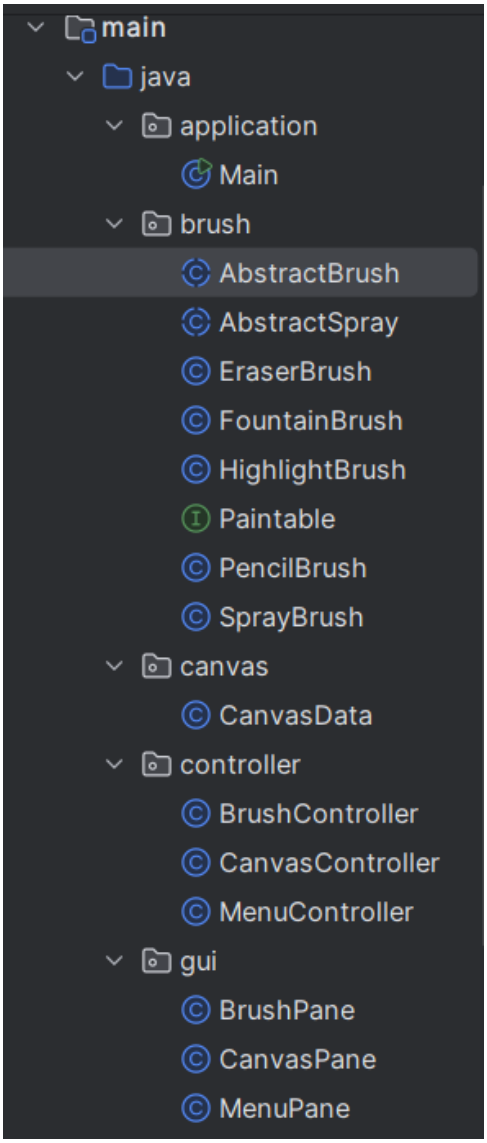
Theme: classic

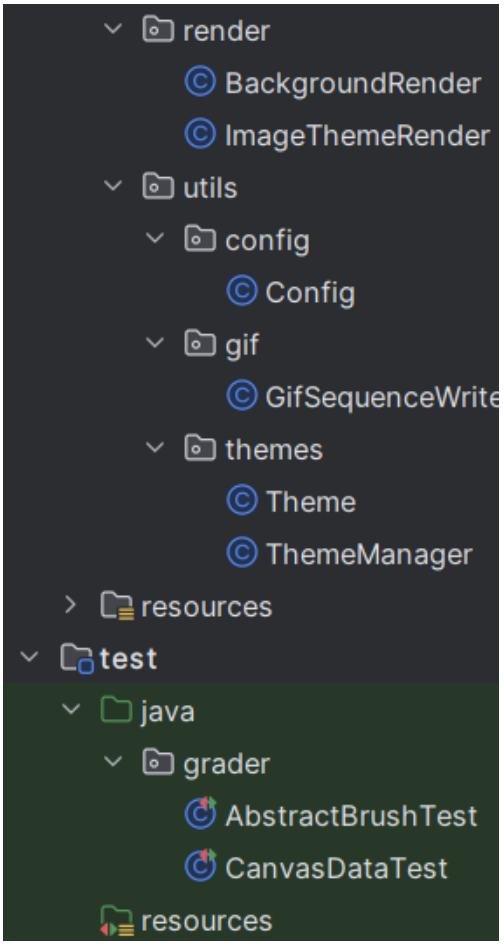


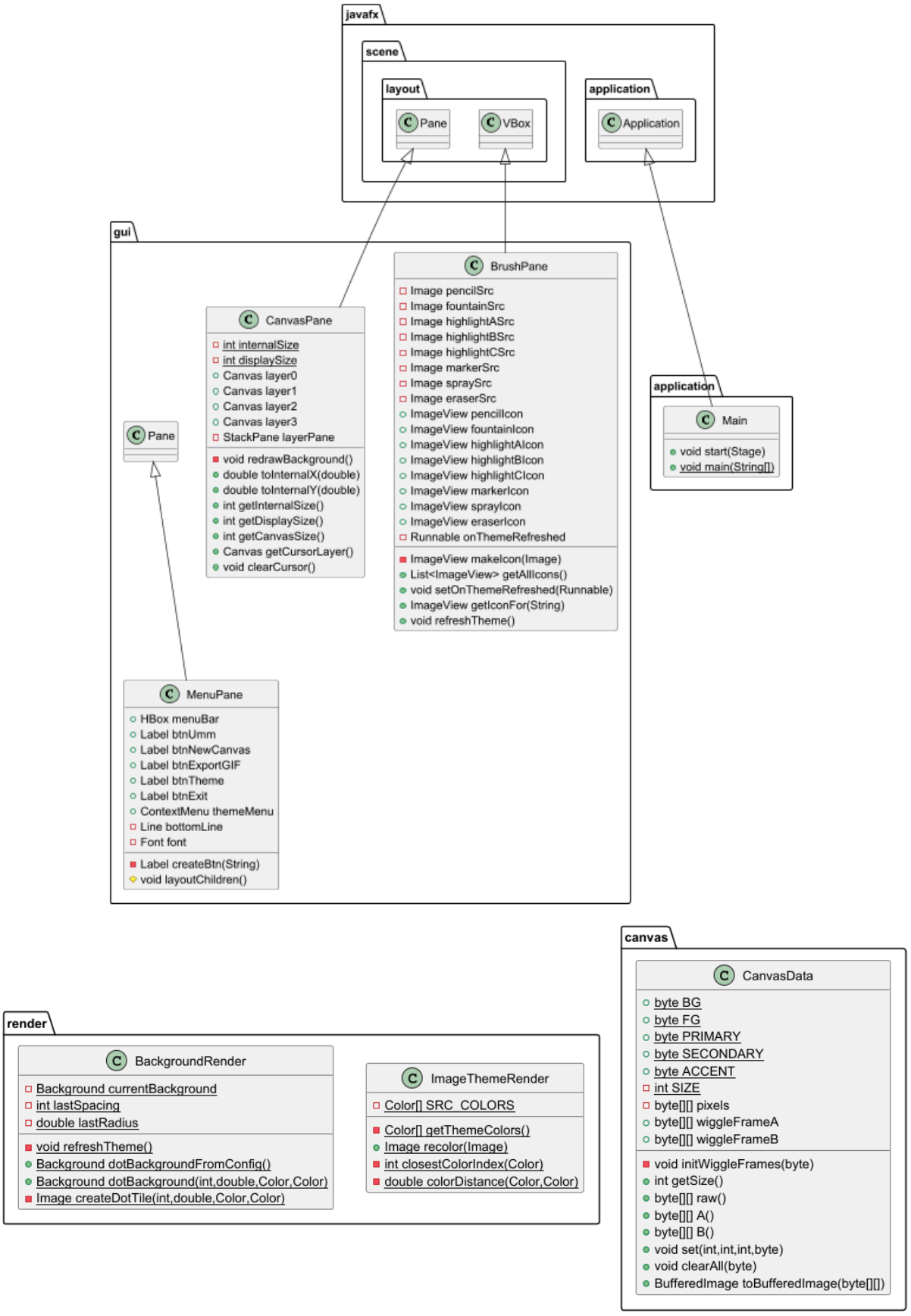
Theme: pastel

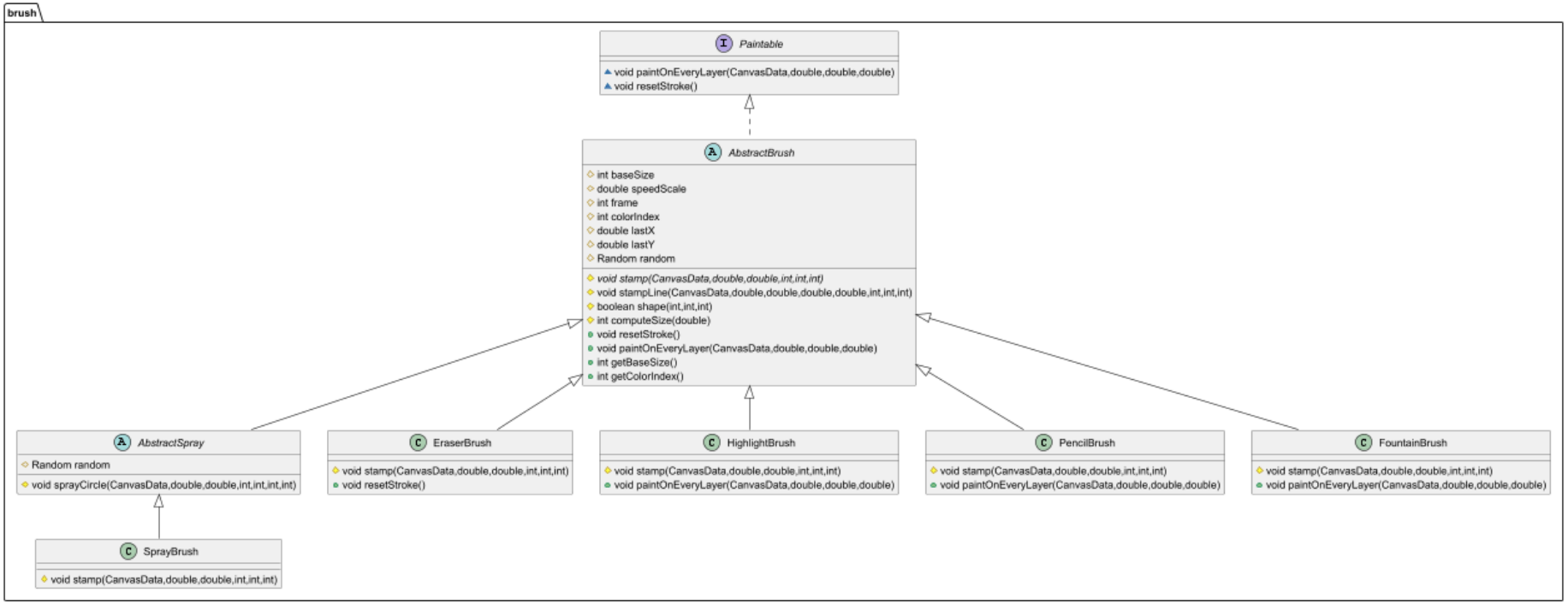
JavaDoc: https://perkiepai.github.io/project-cp-2025-1-umm-docs/

3. Project Structure


main
java
application
Main
brush
AbstractBrush
AbstractSpray
EraserBrush
FountainBrush
HighlightBrush
Paintable
PencilBrush
SprayBrush
canvas
CanvasData
controller
BrushController
CanvasController
MenuController
gui
BrushPane
CanvasPane
MenuPane


render
BackgroundRender
ImageThemeRender
utils
config
Config
gif
GifSequenceWrite
themes
Theme
ThemeManager
resources
test
java
grader
AbstractBrushTest
CanvasDataTest
resources


javafx
scene
layout
Pane
VBox
application
Application
gui
BrushPane
Image pencilSrc
Image fountainSrc
Image highlightASrc
Image highlightBSrc
Image highlightCSrc
Image markerSrc
Image spraySrc
Image eraserSrc
ImageView pencilIcon
ImageView fountainIcon
ImageView highlightAIcon
ImageView highlightBIcon
ImageView highlightCIcon
ImageView markerIcon
ImageView sprayIcon
ImageView eraserIcon
Runnable onThemeRefreshed
ImageView makeIcon(Image)
List<ImageView> getAllIcons()
void setOnThemeRefreshed(Runnable)
ImageView getIconFor(String)
void refreshTheme()
CanvasPane
int internalSize
int displaySize
Canvas layer0
Canvas layer1
Canvas layer2
Canvas layer3
StackPane layerPane
void redrawBackground()
double toInternalX(double)
double toInternalY(double)
int getInternalSize()
int getDisplaySize()
int getCanvasSize()
Canvas getCursorLayer()
void clearCursor()
Pane
MenuPane
HBox menuBar
Label btnUmm
Label btnNewCanvas
Label btnExportGIF
Label btnTheme
Label btnExit
ContextMenu themeMenu
Line bottomLine
Font font
Label createBtn(String)
void layoutChildren()
application
Main
void start(Stage)
void main(String[])
canvas
CanvasData
byte BG
byte FG
byte PRIMARY
byte SECONDARY
byte ACCENT
int SIZE
byte[][] pixels
byte[][] wiggleFrameA
byte[][] wiggleFrameB
void initWiggleFrames(byte)
int getSize()
byte[][] raw()
byte[][] A()
byte[][] B()
void set(int,int,int,byte)
void clearAll(byte)
BufferedImage toBufferedImage(byte[][])
render
BackgroundRender
Background currentBackground
int lastSpacing
double lastRadius
void refreshTheme()
Background dotBackgroundFromConfig()
Background dotBackground(int,double,Color,Color)
Image createDotTile(int,double,Color,Color)
ImageThemeRender
Color[] SRC_COLORS
Color[] getThemeColors()
Image recolor(Image)
int closestColorIndex(Color)
double colorDistance(Color,Color)


brush
Paintable
void paintOnEveryLayer(CanvasData,double,double,double)
void resetStroke()
AbstractBrush
int baseSize
double speedScale
int frame
int colorIndex
double lastX
double lastY
Random random
void stamp(CanvasData,double,double,int,int,int)
void stampLine(CanvasData,double,double,double,double,int,int,int)
boolean shape(int,int,int)
int computeSize(double)
void resetStroke()
void paintOnEveryLayer(CanvasData,double,double,double)
int getBaseSize()
int getColorIndex()
AbstractSpray
Random random
void sprayCircle(CanvasData,double,double,int,int,int,int)
EraserBrush
void stamp(CanvasData,double,double,int,int,int)
void resetStroke()
HighlightBrush
void stamp(CanvasData,double,double,int,int,int)
void paintOnEveryLayer(CanvasData,double,double,double)
PencilBrush
void stamp(CanvasData,double,double,int,int,int)
void paintOnEveryLayer(CanvasData,double,double,double)
FountainBrush
void stamp(CanvasData,double,double,int,int,int)
void paintOnEveryLayer(CanvasData,double,double,double)
SprayBrush
void stamp(CanvasData,double,double,int,int,int)

controller
CanvasController
CanvasPane pane
CanvasData data
BrushController brushController
double lastX
long lastTime
boolean drawing
long lastDrawTime
long DRAG_DELAY_NS
double MIN_DISTANCE
int wiggleFrame
void attachEvents()
void redraw(int)
void drawCursorCircle(double,double,int)
void startWiggleLoop()
BrushController
BrushPane pane
Paintable pencil
Paintable fountain
Paintable marker
Paintable spray
Paintable highlightA
Paintable highlightB
Paintable highlightC
Paintable eraser
Paintable activeBrush
Node currentBrushNode
void attach(Node,Paintable)
void selectBrush(Paintable,Node)
void animateSelect(Node)
void animateBack(Node)
Paintable getActiveBrush()
String getBrushName(Paintable)
void rebindIcons()
MenuController
MenuPane view
CanvasPane canvasPane
CanvasData canvasData
Stage stage
void attachHandlers()
void setupDragWindow()
void setupNewCanvas()
void setupExportGIF()
void setupThemeMenu()
void setupExit()
utils
config
Config
Properties props
void load(String)
String getString(String)
double getDouble(String)
int getInt(String)
themes
Theme
String name
Color bg
Color fg
Color primary
Color secondary
Color accent
Color loadColor(String)
ThemeManager
Theme currentTheme
List<Runnable> listeners
Theme get()
void setTheme(String)
void addListener(Runnable)
gif
GifSequenceWriter
ImageWriter writer
ImageWriteParam params
IIOMetadata metadata
void writeFrame(BufferedImage)
void close()

1. package application

1.1 class Main extends Application

This class is the main entry point of the program. It initializes the main window, layout, canvas, menu, brush panel, controllers, and starts the application.

Methods

Method	Description
+ void start(Stage primaryStage)	Initializes the main UI layout, controllers, and shows the application window.
+ static void main(String[] args)	Launches the JavaFX application.

2. package brush

2.1 class Abstract AbstractBrush implements Paintable

This class represents a base brush used for drawing on the canvas and is inherited by all specific brush types.

Fields

Field	Description
# int baseSize	Base size of the brush.
# double speedScale	Scale factor used to adjust brush size based on speed.
# int colorIndex	Color index used for drawing.
# double lastX	Last X position for continuous stroke drawing.
# double lastY	Last Y position for continuous stroke drawing.
# final Random random	Random generator used for spray particle distribution.

Methods

Method	Description
+ AbstractBrush(int baseSize)	Constructor that sets the base brush size.

+ AbstractBrush(int baseSize, double speedScale)	Constructor that sets base size and speed scale.
+ AbstractBrush(int baseSize, int colorIndex, double speedScale)	Constructor that sets base size, color index, and speed scale.
# abstract void stamp(CanvasData canvas, double x, double y, int size, int colorIndex, int layer)	Abstract method used to draw a single brush stamp.
# void stampLine(CanvasData canvas, double x0, double y0, double x1, double y1, int size, int colorIndex, int layer)	Draws a continuous line between two points using multiple stamps.
# boolean shape(int dx, int dy, int r)	Determines brush shape pattern using random styles.
# int computeSize(double speed)	Computes brush size based on drawing speed.
+ void resetStroke()	Resets the current stroke position.
+ void paintOnEveryLayer(CanvasData canvas, double x, double y, double speed)	Draws the brush on all layers of the canvas.
+ getter/setter	

2.2 class AbstractSpray extends AbstractBrush

This class represents an abstract spray brush that extends AbstractBrush. It provides a common spray behavior by randomly scattering particles inside a circular area. This class cannot be directly instantiated and must be inherited by specific spray brush implementations.

Methods

Method	Description
+ AbstractSpray(int baseSize, double speedScale)	Constructor that initializes the base brush size and speed scale.
# void sprayCircle(CanvasData canvas, double x, double y, int radius, int colorIndex, int layer, int particles)	Draws a spray effect by randomly scattering particles inside a circular area.

2.3 class EraserBrush extends AbstractBrush

This class represents an eraser brush used to remove drawings from the canvas. It erases pixels by drawing over them with the background color. The eraser size does not depend on drawing speed.

Methods

Method	Description

+ EraserBrush(int baseSize)	Constructor that initializes the eraser with a fixed base size and background color.
# void stamp(CanvasData canvas, double x, double y, int size, int colorIndex, int layer)	Erases pixels in a circular area by setting them to the background color.
+ void resetStroke()	Resets the stroke position while keeping continuous erase behavior.

2.4 class FountainBrush extends AbstractBrush

This class represents a fountain brush used for smooth and continuous drawing. It applies a slight random jitter to create a natural ink flow effect.

Methods

Method	Description
+ FountainBrush(int baseSize, double speedScale)	Constructor that initializes the brush with base size and speed scale.
# void stamp(CanvasData canvas, double x, double y, int size, int colorIndex, int layer)	Draws a circular stamp with slight random jitter to simulate fountain pen effect.
+ void paintOnEveryLayer(CanvasData canvas, double x, double y, double speed)	Draws continuously on every layer without broken stroke effect.

2.5 Class HighlightBrush extends AbstractBrush

This class represents a highlight brush used for marking or emphasizing areas on the canvas. It applies a light jitter effect to simulate a hand-drawn highlighting style.

Methods

Method	Description
+ HighlightBrush(int baseSize, int colorIndex, double speedScale)	Constructor that initializes the highlight brush with base size, color, and speed scale.
# void stamp(CanvasData canvas, double x, double y, int size, int colorIndex, int layer)	Draws a highlighted circular area with slight random jitter.
+ void paintOnEveryLayer(CanvasData canvas, double x, double y, double speed)	Draws the highlight continuously on every layer.

2.6 class PencilBrush extends AbstractBrush

This class represents a pencil brush used for natural hand-drawn style drawing. It applies a slight random jitter to simulate the texture of a real pencil.

Methods

Method	Description
+ PencilBrush(int baseSize)	Constructor that initializes the pencil brush with a base size.
# void stamp(CanvasData canvas, double x, double y, int size, int colorIndex, int layer)	Draws a circular pencil stroke with slight random jitter.
+ void paintOnEveryLayer(CanvasData canvas, double x, double y, double speed)	Draws continuously on every layer using the base brush behavior.

2.7 class SprayBrush extends AbstractSpray

This class represents a spray brush that scatters paint particles randomly within a circular area. The density of particles depends on the brush size.

Methods

Method	Description
+ SprayBrush(int baseSize, double speedScale)	Constructor that initializes the spray brush with base size and speed scale.
# void stamp(CanvasData canvas, double x, double y, int size, int colorIndex, int layer)	Applies a spray effect by scattering particles inside a circular area using the inherited spray algorithm.

2.8 interface Paintable

This interface defines the basic behavior for all drawable tools in the application. Any class that implements this interface must support continuous drawing and stroke resetting.

Methods

Method	Description
+ void paintOnEveryLayer(CanvasData canvas, double x, double y, double speed)	Called continuously while the mouse or finger is moving to draw on all canvas layers.
+ void resetStroke()	Resets the current stroke position for a new drawing

	action.

3. package canvas

3.1 CanvasData

This class represents the core pixel buffer of the canvas. It stores all pixel data for the base layer and the two wiggle animation frames, and provides methods for pixel access, clearing, and image export.

Fields

FIeld	Description
+ static final byte BG	Background color index.
+ static final byte FG	Foreground color index.
+ static final byte PRIMARY	Primary theme color index.
+ static final byte SECONDARY	Secondary theme color index.
+ static final byte ACCENT	Accent theme color index.
- static final int SIZE	Canvas size (200 × 200 pixels).
- final byte[][] pixels	Base pixel layer.
+ final byte[][] wiggleFrameA	First wiggle animation frame.
+ final byte[][] wiggleFrameB	Second wiggle animation frame.

Methods

Method	Description
+ CanvasData()	Constructor that initializes the wiggle frames with background color.
- initWiggleFrames(byte color)	Initializes all pixels in both wiggle frames with the given background color.
+ int getSize()	Returns the canvas size.
+ byte[][] raw()	Returns the base pixel layer.
+ byte[][] A()	Returns the first wiggle frame.
+ byte[][] B()	Returns the second wiggle frame.

+ void set(int layeridx, int x, int y, byte index)	Sets a pixel value in the specified layer.
+ void clearAll(byte color)	Clears all layers using the given color.
+ BufferedImage toBufferedImage(byte[][] frame)	Converts a frame buffer into a BufferedImage for export or screenshot.

4. package controller

4.1 class BrushController

This class controls the selection and behavior of all brushes in the application. It binds brush tools to their corresponding icons, handles brush switching, and manages selection animations.

Fields

Field	Description
- final BrushPane pane	Reference to the brush panel UI.
- final Paintable pencil	Pencil brush object.
- final Paintable fountain	Fountain brush object.
- final Paintable marker	Marker brush object.
- final Paintable spray	Spray brush object.
- final Paintable highlightA	Highlight brush A.
- final Paintable highlightB	Highlight brush B.
- final Paintable highlightC	Highlight brush C.
- final Paintable eraser	Eraser brush object.
- Paintable activeBrush	Currently selected brush.
- Node currentBrushNode	Currently selected brush icon node.

Methods

Method	Description
+ BrushController(BrushPane pane)	Constructs the controller and binds all brush icons with their brushes.
- void attach(Node icon, Paintable brush)	Attaches a mouse click event to an icon for selecting a

	brush.
- void selectBrush(Paintable brush, Node icon)	Sets the active brush and updates the UI selection animation.
- void animateSelect(Node node)	Plays the selection animation on the selected brush icon.
- void animateBack(Node node)	Resets the animation of the previously selected brush icon.
+ Paintable getActiveBrush()	Returns the currently selected brush.
- String getBrushName(Paintable b)	Returns the name of a brush as a string.
+ void rebindIcons()	Rebinds all brush icons after a theme change and restores the active brush state.

4.2 class CanvasController

This class controls all user interactions with the canvas. It handles mouse input for drawing, manages rendering of all canvas layers, displays the eraser cursor preview, and controls the wiggle animation.

Fields

Field	Description
- final CanvasPane pane	Reference to the canvas UI pane.
- final CanvasData data	Reference to the canvas pixel buffer.
- final BrushController brushController	Reference to the brush controller.
- double lastX, lastY	Last mouse position for stroke tracking.
- long lastTime	Last timestamp for speed calculation.
- boolean drawing	Indicates whether the user is currently drawing.
- long lastDrawTime	Timestamp for draw throttling.
- static final long DRAG_DELAY_NS	Minimum delay between draw updates.
- static final double MIN_DISTANCE	Minimum distance between stroke points.
- int wiggleFrame	Current wiggle animation frame index.

Methods

Method	Description
+ CanvasController(CanvasPane pane, CanvasData data, BrushController brushController)	Constructs the controller, attaches mouse events, pre-renders wiggle frames, and registers theme refresh behavior.
- void attachEvents()	Attaches mouse press, drag, release, move, and exit events to the canvas input layer.
- void redraw(int layerIndex)	Renders the specified canvas layer to the screen.
- void drawCursorCircle(double x, double y, int radius)	Draws the eraser cursor preview as a circular outline.
+ void startWiggleLoop()	Starts the timeline loop that alternates between wiggle animation frames.

4.3 class MenuController

This class controls all menu-related actions in the application. It handles window dragging, new canvas creation, GIF exporting, theme switching, and application exit.

Fields

Field	Description
- final MenuPane view	Reference to the menu UI.
- final CanvasPane canvasPane	Reference to the canvas pane.
- final CanvasData canvasData	Reference to the canvas pixel data.
- final Stage stage	Reference to the main application stage.

Methods

Method	Description
+ MenuController(MenuPane view, CanvasPane canvasPane, CanvasData canvasData, Stage stage)	Constructs the controller and attaches all menu event handlers.
- void attachHandlers()	Attaches all menu-related event handlers.
- void setupDragWindow()	Enables dragging of the application window using the menu bar.
- void setupNewCanvas()	Clears all layers and resets the canvas to a new blank state.

- void setupExportGIF()	Exports the current wiggle animation as a GIF file.
- void setupThemeMenu()	Handles switching between available application themes.
- void setupExit()	Closes the application when the exit button is clicked.

5. package gui

5.1 class BrushPane extends VBox

This class represents the brush selection panel on the right side of the application. It displays all brush icons, handles theme-based recoloring of icons, and provides access to individual brush icons.

Fields

Field	Description
- Image pencilSrc	Original image source for the pencil icon.
- Image fountainSrc	Original image source for the fountain icon.
- Image highlightASrc	Original image source for highlight A icon.
- Image highlightBSrc	Original image source for highlight B icon.
- Image highlightCSrc	Original image source for highlight C icon.
- Image markerSrc	Original image source for the marker icon.
- Image spraySrc	Original image source for the spray icon.
- Image eraserSrc	Original image source for the eraser icon.
+ ImageView pencilIcon	Themed pencil icon displayed on the UI.
+ ImageView fountainIcon	Themed fountain icon displayed on the UI.
+ ImageView highlightAIcon	Themed highlight A icon displayed on the UI.
+ ImageView highlightBIcon	Themed highlight B icon displayed on the UI.
+ ImageView highlightCIcon	Themed highlight C icon displayed on the UI.
+ ImageView markerIcon	Themed marker icon displayed on the UI.
+ ImageView sprayIcon	Themed spray icon displayed on the UI.
+ ImageView eraserIcon	Themed eraser icon displayed on the UI.
- Runnable onThemeRefreshed	Callback triggered when the theme is refreshed.

Methods

Method	Description
+ BrushPane()	Constructs the brush panel, loads all icons, creates themed icons, and registers a theme change listener.
- ImageView makeIcon(Image raw)	Creates a theme-colored ImageView from a raw source image.
+ List<ImageView> getAllIcons()	Returns a list of all brush icons in the panel.
+ void setOnThemeRefreshed(Runnable callback)	Sets a callback to be executed after the theme is refreshed.
+ ImageView getIconFor(String name)	Returns the corresponding brush icon based on the given brush name.
+ void refreshTheme()	Rebuilds all brush icons using the updated theme and notifies the brush controller.

5.2 class CanvasPane extends Pane

This class represents the main drawing canvas area of the application. It manages multiple layered canvases for background, wiggle animation frames, and cursor input, and handles scaling between internal and display sizes.

Fields

Field	Description
- static final int internalSize	Logical pixel resolution of the canvas (200 × 200).
- static final int displaySize	Display size of the canvas on screen (500 × 500).
+ final Canvas layer0	Background layer (theme-colored).
+ final Canvas layer1	Wiggle animation frame A.
+ final Canvas layer2	Wiggle animation frame B.
+ final Canvas layer3	Cursor and input interaction layer.

- final StackPane layerPane	Container that stacks all canvas layers.

Methods

Method	Description
+ CanvasPane()	Constructs the canvas pane, sets up layered canvases, applies scaling, draws the initial background, and registers a theme refresh listener.
- void redrawBackground()	Redraws the canvas background and border based on the current theme.
+ double toInternalX(double x)	Converts a display X coordinate to an internal canvas coordinate.
+ double toInternalY(double y)	Converts a display Y coordinate to an internal canvas coordinate.
+ int getInternalSize()	Returns the logical internal canvas size.
+ int getDisplaySize()	Returns the display size of the canvas.
+ int getCanvasSize()	Returns the internal canvas size.
+ Canvas getCursorLayer()	Returns the cursor/input layer canvas.
+ void clearCursor()	Clears all drawings from the cursor layer.

5.3 class MenuPane extends Pane

This class represents the top menu bar of the application. It provides user interface controls for creating a new canvas, exporting GIF files, switching themes, and exiting the program. It also handles dynamic theme updates.

Fields

Field	Description
+ final HBox menuBar	Container for all menu buttons.
+ final Label btnUmm	Label button for the main menu title.

+ final Label btnNewCanvas	Label button for creating a new canvas.
+ final Label btnExportGIF	Label button for exporting the canvas as a GIF.
+ final Label btnTheme	Label button for opening the theme menu.
+ final Label btnExit	Label button for exiting the application.
+ final ContextMenu themeMenu	Context menu for selecting application themes.
- final Line bottomLine	Divider line below the menu bar.
- final Font font	Font used for all menu text.

Methods

Method	Description
+ MenuPane()	Constructs the menu bar UI, initializes all buttons, sets up the theme menu, and registers theme update listeners.
- Label createBtn(String text)	Helper method for creating a styled menu button.
# void layoutChildren()	Arranges the menu bar and bottom divider line within the pane.

6. package render

6.1 Class BackgroundRender

This class is responsible for generating and caching the dotted background pattern used in the application. It dynamically updates the background based on theme and configuration settings.

Fields

Field	Description
- static Background currentBackground	Cached background pattern.
- static int lastSpacing	Last used dot spacing value.
- static double lastRadius	Last used dot radius value.

Methods

Method	Description
- static void refreshTheme()	Clears the cached background when the theme changes.
+ static Background dotBackgroundFromConfig()	Returns a dotted background based on configuration and theme values, using cache when possible.
+ static Background dotBackground(int spacing, double dotRadius, Color dotColor, Color bgColor)	Generates a repeating dotted background pattern.
- static Image createDotTile(int size, double dotRadius, Color dotColor, Color bgColor)	Creates a single tile image containing one centered dot.

6.2 Class ImageThemeRender

This class is responsible for recoloring pixel-art icons by mapping predefined source colors to the current theme colors. It ensures that all icons automatically adapt to theme changes.

Fields

Field	Description
- static final Color[] SRC_COLORS	Reference color palette used for matching original pixel colors.

Methods

Method	Description
+ static Color[] getThemeColors()	Returns the current theme color palette from the ThemeManager.
+ static Image recolor(Image src)	Recolors an input image by mapping each pixel to the nearest theme color.
- static int closestColorIndex(Color c)	Finds the nearest reference color index for a given pixel color.
- static double colorDistance(Color a, Color b)	Computes the Euclidean distance between two RGB colors.

7. package utils.config

7.1 class Config

This class manages application configuration values by loading data from .properties files at startup and providing static access methods to retrieve configuration values.

Fields

Field	Description
- static final Properties props	Container that stores all configuration key–value pairs loaded from property files.

Methods

Method	Description
- static void load(String fileName)	Loads a .properties file into the configuration container.
+ static String getString(String key)	Returns a configuration value as a String.
+ static double getDouble(String key)	Returns a configuration value as a double.
+ static int getInt(String key)	Returns a configuration value as an int.

8. package utils.gif

8.1 Class GifSequenceWriter

This class is responsible for creating and writing animated GIF files. It handles GIF metadata configuration, frame delays, looping behavior, and sequential frame writing.

Fields

Field	Description
- final ImageWriter writer	GIF image writer used for output generation.
- final ImageWriteParam params	Parameters used for writing GIF images.
- final IIOMetadata metadata	Metadata used for configuring GIF animation properties.

Methods

Method	Description

+ GifSequenceWriter(ImageOutputStream out, int imageType, int delayMS, boolean loopForever)	Constructs the GIF writer, configures metadata, frame delay, and looping behavior.
+ void writeFrame(BufferedImage img)	Writes a single frame to the GIF sequence.
+ void close()	Finalizes and closes the GIF writing process.

9. package utils.themes

9.1 class Theme

This class represents a color theme of the application. It loads and stores all theme colors from the configuration files based on a given theme name.

Fields

Field	Description
+ final String name	Name of the theme.
+ final Color bg	Background color of the theme.
+ final Color fg	Foreground color of the theme.
+ final Color primary	Primary color of the theme.
+ final Color secondary	Secondary color of the theme.
+ final Color accent	Accent color of the theme.

Methods

Method	Description
+ Theme(String name)	Constructs a theme object and loads all theme colors from configuration files using the given theme name.
- Color loadColor(String key)	Loads a single color value from the configuration file and converts it into a Color object.

9.2 class ThemeManager

This class manages the current theme of the application. It allows switching between themes and notifies all registered listeners to update the user interface when the theme changes.

Fields

Field	Description
- static Theme currentTheme	Current active theme of the application.
- static final List<Runnable> listeners	List of UI listeners that are notified when the theme changes.

Methods

Method	Description
+ static Theme get()	Returns the current active theme.
+ static void setTheme(String name)	Sets a new theme based on the given name and notifies all registered listeners.
+ static void addListener(Runnable uiRefresher)	Registers a UI refresh listener that will be called when the theme changes.